12

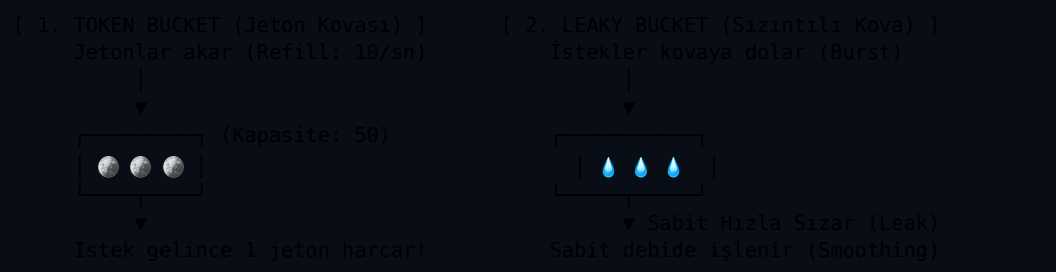
BÖLÜM: YÜK DENGELEME, AĞ PROTOKOLLERI VE API AĞ GEÇITLERI | MODULE: LOAD BALANCING, NETWORKING PROTOCOLS & API GATEWAYS

Dağıtık Hız Sınırlama (Rate Limiting): Token Bucket, Leaky Bucket, Sliding Window

Distributed Rate Limiting: Token Bucket, Leaky Bucket & Sliding Window

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir lunapark trenini düşünün: Trene binebilmek için jeton makinesinden jeton almalısınız (Token Bucket). Makine kovaya her saniye 2 yeni jeton atar; kova en fazla 10 jeton alır. Bir anda 10 kişi gelirse hepsi aynı anda trene biner (Burst). Ama 11. kişi kova dolana kadar kapıda beklemek zorundadır! Rate Limiter işte bu lunapark görevlisidir: Kötü niyetli bir robot veya aşırı heyecanlı bir kullanıcı saniyede 10.000 istek atarak sunucuyu çökertmesin diye 'Sakin ol, kotan doldu! (HTTP 429 Too Many Requests)' der!

🇬🇧 ENGLISH (ELI5)

Imagine an amusement park roller coaster ride requiring tokens (Token Bucket). An automated dispenser drops 2 tokens into a bucket every second, up to a max capacity of 10. If a sudden group of 10 arrives, they grab all 10 tokens and board simultaneously (Burst). But person 11 must wait until new tokens drop! The Rate Limiter is that turnstile guard: shielding databases from aggressive bots or buggy loops by replying with HTTP 429 Too Many Requests!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG mülakatlarında en çok sorulan 4 Rate Limiting algoritması: 1) Token Bucket: Amazon ve Stripe'ın tercihidir. Sabit hızda jeton dolar; burst (ani yük) trafiğe izin verir, bellek tüketimi azdır (sadece sayaç + zaman damgası). 2) Leaky Bucket: Trafiği sabit bir çıkış hızında düzleştirir (Traffic Shaping / Smoothing). 3) Fixed Window Counter: Zamanı sabit pencerelere böler (12:00-12:01); pencere sınırında iki katı istek geçmesine izin veren 'Window Boundary' açığı vardır. 4) Sliding Window Log / Counter: Redis ZSET veya bellek sayaçlarıyla kayan pencereyi milisaniye hassasiyetinde tutar; sınır açığını sıfırlar. Dağıtık ortamda Redis + Lua Scripting ile atomik olarak uygulanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The 4 core rate limiting algorithms evaluated in MAANG rounds: 1) Token Bucket: Favored by Amazon/Stripe. Tokens replenish at a fixed rate; effortlessly absorbs short burst traffic with negligible memory (stores only timestamp + token count). 2) Leaky Bucket: Smooths bursty traffic into a constant outflow rate (vital for throttling queues). 3) Fixed Window Counter: Simple counter reset at window boundaries; vulnerable to 2x burst surges across the window boundary. 4) Sliding Window Counter: Blends historical and current window weights to neutralize boundary spikes with minimal Redis memory overhead. Implemented in distributed clusters via atomic Redis Lua scripts.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Thread-Safe Token Bucket Algoritması
import java.time.Instant;

public class TokenBucketRateLimiter {
    private final long capacity;
    private final double refillTokensPerSecond;
    private double availableTokens;
    private long lastRefillTimestamp;

    public TokenBucketRateLimiter(long capacity, double refillTokensPerSecond) {
        this.capacity = capacity;
        this.refillTokensPerSecond = refillTokensPerSecond;
        this.availableTokens = capacity;
        this.lastRefillTimestamp = System.currentTimeMillis();
    }

    public synchronized boolean tryAcquire() {
        refill();
        if (availableTokens >= 1.0) {
            availableTokens -= 1.0;
            return true; // İstek kabul edildi
        }
        return false; // HTTP 429 Too Many Requests
    }

    private void refill() {
        long now = System.currentTimeMillis();
        double elapsedSeconds = (now - lastRefillTimestamp) / 1000.0;
        availableTokens = Math.min(capacity, availableTokens + elapsedSeconds *
refillTokensPerSecond);
        lastRefillTimestamp = now;
    }
}

void main() {
    var limiter = new TokenBucketRateLimiter(3, 1.0); // 3 kapasite, saniyede 1 jeton
    for (int i = 1; i <= 5; i++) {
        boolean allowed = limiter.tryAcquire();
        IO.println("İstek #" + i + " İzin Verildi mi? " + allowed);
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Dağıtık ortamda her web sunucusunun kendi belleğinde local rate limiter tutması mülakatta eksi puandır! 10 sunucu varsa kullanıcı her sunucuya 100 istek atıp toplamda 1.000 istek yapabilir. Hız sınırlayıcı merkezi bir Redis kümesinde veya Envoy filter seviyesinde tutulmalıdır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Storing rate limit counters in local JVM memory fails in clustered deployments. 10 pods mean a user can fire 10x their quota via round-robin. Centralize distributed counters within a shared Redis cluster utilizing Lua scripts.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Token Bucket algoritması ani yükleri (bursts) tolere ettiği için endüstri standardıdır.

• Dağıtık ortamlarda yarış durumunu (Race Condition) önlemek için Redis Lua Script kullanılır.

• Kotası dolan istemciye HTTP 429 ve 'Retry-After: 30' yanıt başlığı dönülmelidir.

🎯 Key Takeaways for the Interview

• Token Bucket is the industry standard because it gracefully accommodates bursty traffic spikes.

• Redis Lua scripts execute read-and-decrement steps atomically without distributed locks.

• Rate-limited clients must receive HTTP 429 along with a Retry-After header.